

CUDA Kernel 面试背题笔记

LeetCUDA Project

```
1 // =====
2 // notes-v2.cu — CUDA Kernel 面试背题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA), 涵盖:
6 // - 面试高频 CUDA kernel 的完整实现 (共 37 个 kernel)
7 // - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 // - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 // - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 // Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 // Phase 1 — 基础原语: Warp Reduce / Block Reduce (含 broadcast 增强版)
14 // Phase 2 — Elementwise: ReLU / Elementwise Add (基础 + float4 向量化)
15 // Phase 3 — Softmax: naive → safe(2-pass) → online(1-pass) + RMS/Layer Norm
16 // Phase 4 — GEMV: SGEMV K32/K128/K16 + HGEMV K32/K128/K16 (warp-per-row)
17 // Phase 5 — GEMM : SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
18 // Phase 6 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
19 // Phase 7 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
20 // Phase 8 — 杂项: Dot Product / Block All Reduce / Histogram
21 // Phase 9 — FlashAttention split_q (FA-2, 含 online softmax + P&V 寄存器复用)
22 //
23 // =====
24 //
25 // =====
26 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
27 // =====
28 //
29 // ---- GPU 架构速查 ----
30 //
31 // SM (Streaming Multiprocessor) 内部结构:
32 // - Warp Scheduler ×4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
33 // - Register File: 每 SM 65536 × 32-bit = 256KB
34 // - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
35 // - Tensor Cores: Hopper 每 SM 4 个, Blackwell 每 SM 8 个
36 // - Warp = 32 threads: 最小调度单元, SIMT 执行模型
37 //
38 // Memory Hierarchy 带宽数量级 (H100 参考):
39 // HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
40 // L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
41 // L1/SMEM: ~19 TB/s (每 SM ~228KB)
42 // Register: ~0 延迟, ~100+ TB/s 等效带宽
43 //
44 // 关键瓶颈判断:
45 // Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
46 // Compute-bound: AI 足够大, 受限于计算吞吐
47 // Latency-bound: 线程不够多, 无法隐藏内存延迟
48 //
49 // Occupancy 公式:
50 // occupancy = active_warps / max_warps_per_SM
51 // 受限于: register/thread × threads/block + shared_memory/block
52 //
53 // ---- 常见优化手段速查清单 ----
54 //
55 // 1. Coalesced Memory Access (合并访问)
56 // - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
57 // - 否则产生多次事务 (最坏 32 次)
```

```
58 //
59 // 2. Tiling (分块)
60 // - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
61 // - GEMM: Block Tile (BM×BN) + K Tile (BK)
62 //
63 // 3. Vectorized Memory Access (向量化)
64 // - 使用 float4/half2 等向量类型, 减少 load/store 指令数
65 // - float4 = 128-bit, 单条指令加载 16 bytes
66 //
67 // 4. Thread Tile (寄存器分块)
68 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
69 // - 减少线程总数, 降低同步开销
70 //
71 // 5. Bank Conflict Avoidance
72 // - Shared memory 有 32 banks × 4 bytes
73 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
74 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
75 //
76 // 6. Pipeline / Double Buffering (流水线)
77 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
78 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
79 //
80 // 7. Tensor Core (MMA / WGMMMA)
81 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
82 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
83 //
84 // 8. Warp Specialization (Hopper+)
85 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
86 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
87 //
88 // 9. TMA (Tensor Memory Accelerator, Hopper+)
89 // - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
90 // - 配合 cp.async.bulk 实现异步数据搬运
91 //
92 // ---- Roofline 分析公式 ----
93 //
94 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
95 //
96 // GEMM (M=N=K=4096):
97 // FLOPs = 2 × M × N × K = 2 × 4096³ 137 GFLOPS
98 // Bytes = (M×K + K×N + M×N) × sizeof(float) 200 MB
99 // AI 137G / 200M 685 FLOPS/Byte → compute-bound (远超 H100 的 ~50:1)
100 // 墙)
101 //
102 // GEMV (M=4096, K=4096):
103 // FLOPs = 2 × M × K = 2 × 4096² 33 MFLOPS
104 // Bytes = (M×K + K + M) × sizeof(float) 67 MB
105 // AI 33M / 67M 0.5 FLOPS/Byte → severely memory-bound
106 //
107 // Softmax (N=4096): AI (5×N) / (2×N×4) 2.5 FLOPS/Byte → memory-bound
108 //
109 // =====
110 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
111 // =====
112 // 面试要点:
113 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
114 // memory
115 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
116 // 注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
117 // - 为什么不不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
118 //
119 #include <algorithm>
120 #include <cuda_fp16.h>
```

```

121 #include <cuda_runtime.h>
122 #include <float.h>
123 #include <stdio.h>
124 #include <stdlib.h>
125
126 #define WARP_SIZE 32
127 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
128 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
129 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
130
131 // =====
132 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
133 // =====
134
135 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
136 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
137 // 复杂度 O(logN), N=32 时仅需 5 步
138 // 双模板参数: T=数据类型, kWarpSize=segment width (默认 32)
139 // 第四个参数 kWarpSize 是 segment width: 限制 shuffle 在同一 segment 内
140 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
141 template <typename T = float, const int kWarpSize = WARP_SIZE>
142 __device__ __forceinline__ T warp_reduce_sum(T val) {
143     #pragma unroll
144     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
145         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
146     }
147     return val;
148 }
149
150 // Warp Reduce Max — generic
151 template <typename T = float, const int kWarpSize = WARP_SIZE>
152 __device__ __forceinline__ T warp_reduce_max(T val) {
153     #pragma unroll
154     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
155         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
156     }
157     return val;
158 }
159
160 // Warp Reduce Sum — FP16 (保留特化版本, 用于 HGMV)
161 template <const int kWarpSize = WARP_SIZE>
162 __device__ __forceinline__ half warp_reduce_sum_f16(half val) {
163     #pragma unroll
164     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
165         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
166     }
167     return val;
168 }
169
170 // =====
171 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp0)
172 // =====
173
174 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
175 // 两级归约流程:
176 // 1. 每个 warp 内做 warp_reduce_sum → 得到每个 warp 的一个值
177 // 2. warp leader (lane=0) 写入 shared memory
178 // 3. syncthreads 后, lane 0-NUM_WARPS-1 读取 shared memory
179 // 4. warp0 内再做一次 warp_reduce → 得到最终结果
180 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则只有 warp0 知道结果)
181 template <const int NUM_THREADS = 256>
182 __device__ float block_reduce_sum_f32(float val) {
183     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
184     int warp = threadIdx.x / WARP_SIZE;
185     int lane = threadIdx.x % WARP_SIZE;
186     static __shared__ float shared[NUM_WARPS];
187
188     float value = warp_reduce_sum<WARP_SIZE>(val);
189     if (lane == 0)
190         shared[warp] = value;
191     __syncthreads();
192     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
193     value = warp_reduce_sum<NUM_WARPS>(value);
194
195     // 关键: broadcast 结果到所有线程, 后续用 result
196     // 做除法等操作时所有线程都能拿到
197     value = __shfl_sync(0xffffffff, value, 0, 32);
198     return value;
199 }
200
201 // Block Reduce Max — FP32 (增强版, 带 broadcast)
202 template <const int NUM_THREADS = 256>
203 __device__ float block_reduce_max_f32(float val) {
204     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
205     int warp = threadIdx.x / WARP_SIZE;
206     int lane = threadIdx.x % WARP_SIZE;
207     static __shared__ float shared[NUM_WARPS];
208
209     float value = warp_reduce_max<WARP_SIZE>(val);
210     if (lane == 0)
211         shared[warp] = value;
212     __syncthreads();
213     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
214     value = warp_reduce_max<NUM_WARPS>(value);
215     value = __shfl_sync(0xffffffff, value, 0, 32);
216     return value;
217 }
218
219 // Block Reduce Sum — FP32 (notes-v1 原版, 保留用于兼容旧 kernel)
220 // 注意: 此版本没有 broadcast, 仅 warp0 持有最终结果
221 template <const int NUM_THREADS = 128>
222 __device__ __forceinline__ float block_reduce_sum(float val) {
223     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
224     int warp = threadIdx.x / WARP_SIZE;
225     int lane = threadIdx.x % WARP_SIZE;
226     static __shared__ float shared[NUM_WARPS];
227
228     val = warp_reduce_sum<WARP_SIZE>(val);
229     if (lane == 0)
230         shared[warp] = val;
231     __syncthreads();
232     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
233     val = warp_reduce_sum<NUM_WARPS>(val);
234     return val;
235 }
236
237 template <const int NUM_THREADS = 128>
238 __device__ __forceinline__ float block_reduce_max(float val) {
239     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
240     int warp = threadIdx.x / WARP_SIZE;
241     int lane = threadIdx.x % WARP_SIZE;
242     static __shared__ float shared[NUM_WARPS];
243
244     val = warp_reduce_max<WARP_SIZE>(val);
245     if (lane == 0)
246         shared[warp] = val;
247     __syncthreads();
248     val = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
249     val = warp_reduce_max<NUM_WARPS>(val);
250     return val;
251 }
252
253 // =====
254 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
255 // =====
256
257 // 面试要点:
258 // - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
259 // - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
260 // - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
261
262 // ---- ReLU: y = max(0, x) ----
263 // Grid: ((N + 255) / 256, 1, 1)
264 // Block: (256, 1, 1)
265 // source: LeetCUDA/kernels/relu/relu.cu
266 __global__ void relu(float *x, float *y, int N) {
267     int idx = blockIdx.x * blockDim.x + threadIdx.x;
268     if (idx < N)
269         y[idx] = fmaxf(0.0f, x[idx]);
270 }

```

```

268 }
269
270 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
271 // Block: (256, 1, 1)
272 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
273 // Grid: ((N + 255) / 256, 1, 1)
274 // Block: (64, 1, 1)
275 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
276 // source: LeetCode/kernels/relu/relu.cu
277 __global__ void relu_vec4(float *x, float *y, int N) {
278     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
279     if (idx < N) {
280         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
281         float4 reg_y;
282         reg_y.x = fmaxf(0.0f, reg_x.x);
283         reg_y.y = fmaxf(0.0f, reg_x.y);
284         reg_y.z = fmaxf(0.0f, reg_x.z);
285         reg_y.w = fmaxf(0.0f, reg_x.w);
286         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
287     }
288 }
289
290 // ---- Elementwise Add: c = a + b ----
291 // Grid: ((N + 255) / 256, 1, 1)
292 // Block: (256, 1, 1)
293 // source: LeetCode/kernels/elementwise/elementwise.cu
294 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
295     int idx = blockIdx.x * blockDim.x + threadIdx.x;
296     if (idx < N)
297         c[idx] = a[idx] + b[idx];
298 }
299
300 // Elementwise Add + float4 向量化
301 // Grid: ((N + 255) / 256, 1, 1)
302 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
303 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
304 // source: LeetCode/kernels/elementwise/elementwise.cu
305 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
306     {
307         int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
308         if ((idx + 3) < N) {
309             float4 reg_a = FLOAT4(a[idx]);
310             float4 reg_b = FLOAT4(b[idx]);
311             float4 reg_c;
312             reg_c.x = reg_a.x + reg_b.x;
313             reg_c.y = reg_a.y + reg_b.y;
314             reg_c.z = reg_a.z + reg_b.z;
315             reg_c.w = reg_a.w + reg_b.w;
316             FLOAT4(c[idx]) = reg_c;
317         } else if (idx < N) {
318             for (int i = 0; (idx + i) < N; i++) {
319                 c[idx + i] = a[idx + i] + b[idx + i];
320             }
321         }
322     }
323 // =====
324 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
325 // =====
326 // 面试要点:
327 // - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
328 //   online (1-pass, 增量更新)
329 // - Online Softmax 是 FlashAttention 的数学基础
330 // - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次
331 //   (mean
332 // + variance)
333 // - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
334 // ---- Online Softmax 辅助结构 ----
335 // MD struct: 存储 running max (m) 和 running denominator (d)
336 // 算法来源: "Online normalizer calculation for softmax" (arXiv
337 // :1805.02867)
338 // 核心递推公式:
339 // m_new = max(m_old, m_cur)
340 // d_new = d_old * exp(m_old - m_new) + d_cur * exp(m_cur - m_new)
341 // FlashAttention 中用同样的公式做 online rescaling:
342 // O_new = diag(exp(m_old - m_new)) * O_old + exp(m_cur - m_new) * P@V
343 struct __align__(8) MD {
344     float m; // running max
345     float d; // running denominator (sum of exp(x - max))
346 };
347 // Warp Reduce for Online Softmax
348 // 与普通 reduce 不同: 归约时需同时更新 m 和 d (因为 max 在不断变大)
349 template <const int kWarpSize = WARP_SIZE>
350 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
351     unsigned int mask = 0xffffffff;
352 #pragma unroll
353     for (int stride = kWarpSize >> 1; stride >= 1; stride >>= 1) {
354         MD other;
355         other.m = __shfl_xor_sync(mask, value.m, stride);
356         other.d = __shfl_xor_sync(mask, value.d, stride);
357
358         bool value_bigger = (value.m > other.m);
359         MD bigger_m = value_bigger ? value : other;
360         MD smaller_m = value_bigger ? other : value;
361
362         // 关键: 更新 d 时需要 rescale 旧的 d 到新的 max 尺度下
363         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
364         value.m = bigger_m.m;
365     }
366     return value;
367 }
368
369 // =====
370 // Phase 3a: Softmax — 三级递进 (面试核心考点)
371 // =====
372 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
373 // 答案线索: naive(溢出) → safe(2-pass, 稳定) → online(1-pass, 为 FA 打基础)
374
375 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定)----
376 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
377 // 问题: x 值很大时 exp(x) 溢出为 inf
378 template <const int NUM_THREADS = 256>
379 // Grid: (S, 1, 1), S=batch*seq_len,
380 // DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
381 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个
382 // block
383 // 处理一个 token
384 // source: LeetCode/kernels/softmax/softmax.cu
385 __global__ void softmax_f32_per_token_kernel(float *x, float *y, int N) {
386     const int tid = threadIdx.x;
387     const int idx = blockIdx.x * blockDim.x + tid;
388
389     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
390     float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);
391     if (idx < N)
392         y[idx] = exp_val / exp_sum;
393 }
394
395 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定)----
396 // 面试重点: 为什么 Softmax 需要 Safe?
397 // - exp(89) 4.5e38 已接近 float32 上限, exp(100) 直接 inf
398 // - 减去 max 后: exp(x - max) exp(0) = 1.0, 永不超 1
399 // - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
400 // - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
401 template <const int NUM_THREADS = 256>
402 // Grid: (S, 1, 1)
403 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
404 // source: LeetCode/kernels/softmax/softmax.cu
405 __global__ void safe_softmax_f32_per_token_kernel(float *x, float *y, int
406     N) {
407     const int tid = threadIdx.x;
408     const int idx = blockIdx.x * blockDim.x + tid;
409
410     // Pass 1: block reduce max — 找最大值
411     float val = (idx < N) ? x[idx] : -FLT_MAX;
412     float max_val = block_reduce_max_f32<NUM_THREADS>(val);

```

```

410
411 // Pass 2: exp(x - max) → block reduce sum
412 float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
413 float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);
414
415 if (idx < N)
416     y[idx] = exp_val / exp_sum;
417 }
418
419 // ---- Level 3: Online Safe Softmax (1-pass, FlashAttention 的数学基础)
420 // ----
421 // 面试重点: Online Softmax 为什么重要?
422 // - Safe Softmax 需要 3 遍遍历: 找 max + exp+sum → 除法
423 // - Online Softmax 用递推公式合并为 1 遍遍历
424 // - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
425 // - 正是 FlashAttention 中 "online rescaling":
426 // 0_new = diag(exp(m_old - m_new)) * 0_old + exp(m_cur - m_new) * P@V
427 // 算法参考: "Online normalizer calculation for softmax" (arXiv
428 // :1805.02867)
429 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会
430 // 写回 y
431 template <const int NUM_THREADS = 256>
432 // Grid: (S, 1, 1)
433 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
434 // source: LeetCUDA/kernels/softmax/softmax.cu
435 __global__ void online_safe_softmax_f32_per_token_kernel(const float *x,
436 float *y, int N)
437 {
438     int local_tid = threadIdx.x;
439     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
440     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
441     int warp_id = local_tid / WARP_SIZE;
442     int lane_id = local_tid % WARP_SIZE;
443
444     // 初始化: 每个线程持有一个 (max, denom) 对
445     MD val;
446     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
447     val.d = global_tid < N ? 1.0f : 0.0f;
448
449     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
450     __shared__ MD shared[WARP_NUM];
451     MD res = warp_reduce_md_op<WARP_SIZE>(val);
452
453     if (lane_id == 0)
454         shared[warp_id] = res;
455     __syncthreads();
456
457     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模
458     // 式)
459     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才
460     // 对整个
461     // block 可见
462     if (local_tid < WARP_SIZE) {
463         MD block_res =
464             local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
465         block_res = warp_reduce_md_op<WARP_NUM>(block_res);
466         if (local_tid == 0) {
467             shared[0] = block_res;
468         }
469     }
470     __syncthreads();
471
472     // 用全局 max 和 denom 做最终 softmax
473     MD final_res = shared[0];
474     float d_total_inverse = __fdivdef(1.0f, final_res.d);
475     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
476     if (global_tid < N) {
477         y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse
478             ;
479     }
480 }
481
482 // 面试要点:
483 // - RMS Norm: y = x / rms(x) * g, rms(x) = sqrt(mean(x^2))
484 // - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
485 // - Llama 系列使用 RMS Norm
486 // - grid(N, K/K), block(K): 一行一个 block
487
488 template <const int NUM_THREADS = 128>
489 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
490 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
491 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
492 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
493     int tid = threadIdx.x;
494     int bid = blockIdx.x;
495     int idx = bid * blockDim.x + threadIdx.x;
496     const float epsilon = 1e-5f;
497
498     __shared__ float s_variance;
499     float value = (idx < N * K) ? x[idx] : 0.0f;
500     float variance = value * value;
501     variance = block_reduce_sum<NUM_THREADS>(variance);
502     if (tid == 0)
503         s_variance = rsqrtf(variance / (float)K + epsilon);
504     __syncthreads();
505     if (idx < N * K)
506         y[idx] = (value * s_variance) * g;
507 }
508
509 // RMS Norm + float4
510 template <const int NUM_THREADS = 128 / 4>
511 // Grid: (N, 1, 1)
512 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处
513 // 理
514 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
515 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
516 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K)
517 {
518     int tid = threadIdx.x;
519     int bid = blockIdx.x;
520     int idx = (bid * blockDim.x + threadIdx.x) * 4;
521     const float epsilon = 1e-5f;
522
523     __shared__ float s_variance;
524     float4 reg_x = FLOAT4(x[idx]);
525     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y
526         +
527         reg_x.z * reg_x.z + reg_x.w * reg_x.w
528         )
529         : 0.0f;
530     variance = block_reduce_sum<NUM_THREADS>(variance);
531     if (tid == 0)
532         s_variance = rsqrtf(variance / (float)K + epsilon);
533     __syncthreads();
534     float4 reg_y;
535     reg_y.x = reg_x.x * s_variance * g;
536     reg_y.y = reg_x.y * s_variance * g;
537     reg_y.z = reg_x.z * s_variance * g;
538     reg_y.w = reg_x.w * s_variance * g;
539     if (idx < N * K)
540         FLOAT4(y[idx]) = reg_y;
541 }
542
543 // =====
544 // Phase 3c: Layer Normalization (2-pass reduce)
545 // =====
546 // 面试要点:
547 // - Layer Norm: y = (x - mean) / std * g + b
548 // - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/
549 // K)
550 // - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算
551 // variance
552
553 template <const int NUM_THREADS = 128>
554 // Grid: (N, 1, 1), 一行一个 block
555 // Block: (128, 1, 1), NUM_THREADS=K=128
556 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu

```



```

547 __global__ void layer_norm(float *x, float *y, float g, float b, int N,
    int K) {
548     int tid = threadIdx.x;
549     int bid = blockIdx.x;
550     int idx = bid * blockDim.x + threadIdx.x;
551     const float epsilon = 1e-5f;
552
553     __shared__ float s_mean;
554     __shared__ float s_variance;
555     float value = (idx < N * K) ? x[idx] : 0.0f;
556
557     // Pass 1: compute mean
558     float sum = block_reduce_sum<NUM_THREADS>(value);
559     if (tid == 0)
560         s_mean = sum / (float)K;
561     __syncthreads(); // 必须等待 s_mean 对所有线程可见
562
563     // Pass 2: compute variance = (x - mean)^2
564     float variance = (value - s_mean) * (value - s_mean);
565     variance = block_reduce_sum<NUM_THREADS>(variance);
566     if (tid == 0)
567         s_variance = rsqrtf(variance / (float)K + epsilon);
568     __syncthreads(); // 必须等待 s_variance 对所有线程可见
569
570     if (idx < N * K)
571         y[idx] = ((value - s_mean) * s_variance) * g + b;
572 }
573
574 // Layer Norm + float4
575 template <const int NUM_THREADS = 128 / 4>
576 // Grid: (N, 1, 1)
577 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处
    理
578 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
579 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
580 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int
    N,
    int K) {
581     int tid = threadIdx.x;
582     int bid = blockIdx.x;
583     int idx = (bid * blockDim.x + threadIdx.x) * 4;
584     const float epsilon = 1e-5f;
585
586     __shared__ float s_mean;
587     __shared__ float s_variance;
588     float4 reg_x = FLOAT4(x[idx]);
589     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w)
        : 0.0f;
590
591     float sum = block_reduce_sum<NUM_THREADS>(value);
592     if (tid == 0)
593         s_mean = sum / (float)K;
594     __syncthreads();
595
596     float4 reg_x_hat;
597     reg_x_hat.x = reg_x.x - s_mean;
598     reg_x_hat.y = reg_x.y - s_mean;
599     reg_x_hat.z = reg_x.z - s_mean;
600     reg_x_hat.w = reg_x.w - s_mean;
601     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y
        +
        reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
602     variance = block_reduce_sum<NUM_THREADS>(variance);
603     if (tid == 0)
604         s_variance = rsqrtf(variance / (float)K + epsilon);
605     __syncthreads();
606
607     float4 reg_y;
608     reg_y.x = reg_x_hat.x * s_variance * g + b;
609     reg_y.y = reg_x_hat.y * s_variance * g + b;
610     reg_y.z = reg_x_hat.z * s_variance * g + b;
611     reg_y.w = reg_x_hat.w * s_variance * g + b;
612     if (idx < N * K)
613         FLOAT4(y[idx]) = reg_y;
614 }
615
616
617
618 // =====
619 // Phase 4: GEMV — 矩阵向量乘 (纯 memory-bound 算子, warp-per-row 策略)
620 // =====
621 // 面试要点:
622 // - GEMV 是典型的 memory-bound 算子: AI 0(1), 瓶颈在内存带宽
623 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
624 // - 不同 K 值对应不同分块策略:
625 // K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
626 // K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
627 // K=16 < 32: 一个 warp 不够, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
628 //
629 // a: MxK, x: Kx1, y: Mx1, 计算: y = a * x
630
631 // ---- SGEMV K32: 基础 warp-per-row ----
632 // 设计: block(32, 4), blockDim.x=WARP_SIZE=K, blockDim.y=4 行/block
633 // grid(M/4), 每个 warp 负责一行
634 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
635 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
636 // Block: (32, 4, 1), 每行 1 warp
637 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
638 // source: LeetCUDA/kernels/sgemv/sgemv.cu
639 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
640     int tx = threadIdx.x; // 0-31
641     int ty = threadIdx.y; // 0-3
642     int bx = blockIdx.x;
643     int lane = tx % WARP_SIZE; // 0-31
644     int m = bx * blockDim.y + ty; // 全局行号
645     if (m < M) {
646         float sum = 0.0f;
647         int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
648         #pragma unroll
649         for (int w = 0; w < NUM_WARPS; ++w) {
650             int k = w * WARP_SIZE + lane;
651             sum += a[m * K + k] * x[k];
652         }
653         sum = warp_reduce_sum<WARP_SIZE>(sum);
654         if (lane == 0)
655             y[m] = sum;
656     }
657 }
658
659 // ---- SGEMV K128: float4 向量化 ----
660 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
661 // Grid: ((M + 3) / 4, 1, 1)
662 // Block: (32, 4, 1)
663 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
664 // source: LeetCUDA/kernels/sgemv/sgemv.cu
665 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
666     int tx = threadIdx.x; // 0-31
667     int ty = threadIdx.y; // 0-3
668     int bx = blockIdx.x;
669     int lane = tx % WARP_SIZE; // 0-31
670     int m = blockDim.y * bx + ty;
671
672     if (m < M) {
673         float sum = 0.0f;
674         // K/128 个 warp 覆盖 K 维
675         int NUM_WARPS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
676         #pragma unroll
677         for (int w = 0; w < NUM_WARPS; ++w) {
678             int k = (w * WARP_SIZE + lane) * 4;
679             float4 reg_x = FLOAT4(x[k]);
680             float4 reg_a = FLOAT4(a[m * K + k]);
681             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
                reg_a.w * reg_x.w);
682         }
683         sum = warp_reduce_sum<WARP_SIZE>(sum);
684         if (lane == 0)
685             y[m] = sum;
686     }
687 }
688 }
689
690 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
691 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行

```

```

692 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理
        处理
693 // row1
694 template <const int ROW_PER_WARP = 2>
695 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
696 // Block: (32, 4, 1)
697 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理
        2 行
698 // source: LeetCUDA/kernels/sgemv/sgemv.cu
699 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
700     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) /
        ROW_PER_WARP;
701     int tx = threadIdx.x;
702     int ty = threadIdx.y;
703     int bx = blockIdx.x;
704     int lane = tx % WARP_SIZE;
705     int k = lane % K_WARP_SIZE; // 0-15
706     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
707     if (m < M) {
708         float sum = A[m * K + k] * x[k];
709         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
710         // 注意: 判断条件是 k == 0, 不是 lane == 0!
711         if (k == 0)
712             y[m] = sum;
713     }
714 }
715
716 // ---- HGEMV K32: FP16 版本 ----
717 // 策略与 SGEMV K32 完全一致, 仅数据类型从 float 变为 half
718 // Grid: ((M + 3) / 4, 1, 1)
719 // Block: (32, 4, 1)
720 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
721 // source: LeetCUDA/kernels/hgemv/hgemv.cu
722 __global__ void hgemv_k32_f16_kernel(half *a, half *x, half *y, int M,
        int K) {
723     int tx = threadIdx.x;
724     int ty = threadIdx.y;
725     int bx = blockIdx.x;
726     int lane = tx % WARP_SIZE;
727     int m = bx * blockDim.y + ty;
728     if (m < M) {
729         half sum = 0.0f;
730         int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
731 #pragma unroll
732         for (int w = 0; w < NUM_WARPS; ++w) {
733             int k = w * WARP_SIZE + lane;
734             sum += a[m * K + k] * x[k];
735         }
736         sum = warp_reduce_sum_f16<WARP_SIZE>(sum); // FP16 warp reduce
737         if (lane == 0)
738             y[m] = sum;
739     }
740 }
741
742 // ---- HGEMV K128: FP16 + half2 向量化 ----
743 // 每个线程处理 4 个 half (2 个 half2), 一个 warp 覆盖 128 个元素
744 // Grid: ((M + 3) / 4, 1, 1)
745 // Block: (32, 4, 1)
746 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 half2 打包访问前提
747 // source: LeetCUDA/kernels/hgemv/hgemv.cu
748 __global__ void hgemv_k128_f16x4_kernel(half *a, half *x, half *y, int M,
        int K) {
749     int tx = threadIdx.x;
750     int ty = threadIdx.y;
751     int bx = blockIdx.x;
752     int lane = tx % WARP_SIZE;
753     int m = blockDim.y * bx + ty;
754
755     if (m < M) {
756         half sum = 0.0f;
757         int NUM_WARPS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
758 #pragma unroll
759         for (int w = 0; w < NUM_WARPS; ++w) {
760             int k = (w * WARP_SIZE + lane) * 4;
761             half2 reg_x_0 = HALF2(x[k + 0]);
762
763             half2 reg_x_1 = HALF2(x[k + 2]);
764             half2 reg_a_0 = HALF2(a[m * K + k + 0]);
765             half2 reg_a_1 = HALF2(a[m * K + k + 2]);
766             sum += (reg_x_0.x * reg_a_0.x + reg_x_0.y * reg_a_0.y +
767                 reg_x_1.x * reg_a_1.x + reg_x_1.y * reg_a_1.y);
768         }
769         sum = warp_reduce_sum_f16<WARP_SIZE>(sum);
770         if (lane == 0)
771             y[m] = sum;
772     }
773 }
774
775 // ---- HGEMV K16: FP16 + ROW_PER_WARP=2 ----
776 template <const int ROW_PER_WARP = 2>
777 // Grid: ((M + 7) / 8, 1, 1)
778 // Block: (32, 4, 1)
779 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理
        2 行
780 // source: LeetCUDA/kernels/hgemv/hgemv.cu
781 __global__ void hgemv_k16_f16_kernel(half *A, half *x, half *y, int M,
        int K) {
782     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) /
        ROW_PER_WARP;
783     int tx = threadIdx.x;
784     int ty = threadIdx.y;
785     int bx = blockIdx.x;
786     int lane = tx % WARP_SIZE;
787     int k = lane % K_WARP_SIZE;
788     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
789     if (m < M) {
790         half sum = A[m * K + k] * x[k];
791         sum = warp_reduce_sum_f16<K_WARP_SIZE>(sum);
792         if (k == 0)
793             y[m] = sum;
794     }
795 }
796
797 // =====
798 // Phase 5: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
799 // =====
800 // 面试要点 (GEMM 优化五层金字塔):
801 // Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
802 // Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
803 // 个元素, 提高计算密度
804 // Level 3 — Vectorize (向量化访存): float4/half2, 减少
805 // load/store 指令数
806 // Level 4 — Tensor Core (MMA
807 // m16n8k16): 硬件矩阵乘单元, warp 级指令
808 // Level 5 — Warp Specialization +
809 // TMA (WGMMMA m64n128k16): Hopper 异步执行
810 //
811 // 计算密度递进:
812 // Level 1: AI B_K / (2*sizeof) 32/8 = 4 → 仍是 memory-bound
813 // Level 2: AI TM×TN×B_K / (2*sizeof) 8×8×8/8 = 64 → compute-bound
814 // Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
815
816 // =====
817 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
818 // =====
819
820 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
821 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
822 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
823 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
824 // Block: (32, 32, 1)
825 // source: LeetCUDA/kernels/sgemm/sgemm.cu
826 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K)
        {
827     constexpr int BM = 32;
828     constexpr int BN = 32;
829     constexpr int BK = 32;
830     __shared__ float s_a[BM][BK], s_b[BK][BN];
831
832     int bx = blockIdx.x;
833     int by = blockIdx.y;

```

```

834 int tx = threadIdx.x;
835 int ty = threadIdx.y;
836 int tid = threadIdx.y * blockDim.x + tx;
837
838 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
839 int load_smem_a_m = tid / 32;
840 int load_smem_a_k = tid % 32;
841 int load_smem_b_k = tid / 32;
842 int load_smem_b_n = tid % 32;
843 int load_gmem_a_m = by * BM + load_smem_a_m;
844 int load_gmem_b_n = bx * BN + load_smem_b_n;
845
846 float sum = 0.f;
847 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
848     int load_gmem_a_k = bk * BK + load_smem_a_k;
849     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
850     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
851     int load_gmem_b_k = bk * BK + load_smem_b_k;
852     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
853     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
854     __syncthreads(); // 确保整个 smem tile 加载完毕
855
856 #pragma unroll
857     for (int k = 0; k < BK; ++k) {
858         int comp_smem_a_m = load_smem_a_m;
859         int comp_smem_b_n = load_smem_b_n;
860         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
861     }
862     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
863 }
864 int store_gmem_c_m = load_gmem_a_m;
865 int store_gmem_c_n = load_gmem_b_n;
866 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
867 c[store_gmem_c_addr] = sum;
868 }
869
870 // ---- Level 2+3: SGEMM — Block Tile 128x128 + Thread Tile 8x8 + float4
871 // ----
872 // 三层 tile hierarchy: Block Tile → Thread Tile → K Tile
873 // BM=BN=128, TM=TN=8, BK=8, block(16, 16)=256 threads
874 // 每个线程计算 TM*TN=64 个元素, 大幅提升计算密度
875 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
876 // Block: (16, 16, 1), 256 线程
877 // source: LeetCUDA/kernels/sgemm/sgemm.cu
878 __global__ void sgemm_thread_tile_vec4(float *a, float *b, float *c, int
M,
int N, int K) {
879     constexpr int BM = 128;
880     constexpr int BN = 128;
881     constexpr int BK = 8;
882     constexpr int TM = 8;
883     constexpr int TN = 8;
884
885     int bx = blockIdx.x;
886     int by = blockIdx.y;
887     int tx = threadIdx.x;
888     int ty = threadIdx.y;
889     int tid = threadIdx.y * blockDim.x + tx;
890
891     // smem: 2x128x8x4 = 8KB (只有 1/4 的 L1/SMEM!)
892     __shared__ float s_a[BM][BK], s_b[BK][BN];
893
894     // 线程到 smem 的映射 — 256 线程协作加载 128x8 的矩阵
895     // s_a[BM][BK]=[128][8]: 每行 8 元素, 2 线程/行(float4)→ 256 线程覆盖
896     // 128 行
897     int load_smem_a_m = tid / 2;
898     int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
899     // s_b[BK][BN]=[8][128]: 每行 128 元素, 32 线程/行(float4)→ 256 线程覆盖
900     // 8 行
901     int load_smem_b_k = tid / 32;
902     int load_smem_b_n = (tid % 32) * 4;
903
904     int load_gmem_a_m = by * BM + load_smem_a_m;
905     int load_gmem_b_n = bx * BN + load_smem_b_n;
906
907     // 寄存器分块: 每个线程累积 TM*TN=64 个部分和
908     float r_c[TM][TN] = {0.0};
909
910     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
911         int load_gmem_a_k = bk * BK + load_smem_a_k;
912         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
913         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr
]);
914
915         int load_gmem_b_k = bk * BK + load_smem_b_k;
916         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
917         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr
]);
918         __syncthreads();
919
920         #pragma unroll
921         for (int k = 0; k < BK; k++) {
922             #pragma unroll
923             for (int m = 0; m < TM; m++) {
924                 #pragma unroll
925                 for (int n = 0; n < TN; n++) {
926                     int comp_smem_a_m = ty * TM + m;
927                     int comp_smem_b_n = tx * TN + n;
928                     r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
929                 }
930             }
931             __syncthreads();
932         }
933
934         // 写回: float4 向量化 store
935         #pragma unroll
936         for (int m = 0; m < TM; ++m) {
937             int store_gmem_c_m = by * BM + ty * TM + m;
938             #pragma unroll
939             for (int n = 0; n < TN; n += 4) {
940                 int store_gmem_c_n = bx * BN + tx * TN + n;
941                 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
942                 FLOAT4(c[store_gmem_c_addr]) = FLOAT4(r_c[m][n]);
943             }
944         }
945     }
946
947     // ---- Level 2+3 (FP16): HGEMM — Thread Tile 8x8 + half2 向量化 ----
948     // FP16 版本: BM=BN=128, TM=TN=8, BK=8, block(16, 16)
949     // half2 向量化: 每次加载 2 个 half (32-bit) 到寄存器
950     // 面试注意: 这里的优化重点是 half2 访存向量化; 算术语义以当前 half 运算符实
951     // 现为准
952     template <const int BM = 128, const int BN = 128, const int BK = 8,
953             const int TM = 8, const int TN = 8>
954     // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
955     // Block: (16, 16, 1), 256 线程
956     // source: LeetCUDA/kernels/hgemm/naive/hgemm.cu
957     __global__ void hgemm_t_8x8_sliced_k_f16x4_kernel(half *a, half *b, half
*c,
int M, int N, int K) {
958         int bx = blockIdx.x;
959         int by = blockIdx.y;
960         int tx = threadIdx.x;
961         int ty = threadIdx.y;
962         int tid = threadIdx.y * blockDim.x + tx;
963
964         // FP16 smem: 2x128x8x2 = 4KB (是 FP32 的一半!)
965         __shared__ half s_a[BM][BK], s_b[BK][BN];
966
967         // 线程到 smem 映射与 FP32 版本相同
968         int load_smem_a_m = tid / 2;
969         int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
970         int load_smem_b_k = tid / 32;
971         int load_smem_b_n = (tid % 32) * 4;
972
973         int load_gmem_a_m = by * BM + load_smem_a_m;
974         int load_gmem_b_n = bx * BN + load_smem_b_n;
975
976         // 注意: 寄存器分块用 half 类型

```

```

976 half r_c[TM][TN] = {__float2half(0.0f)};
977
978 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
979     int load_gmem_a_k = bk * BK + load_smem_a_k;
980     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
981     // half2 向量化: 一次加载 2 个 half = 4 bytes
982     HALF2(s_a[load_smem_a_m][load_smem_a_k]) = HALF2(a[load_gmem_a_addr
983     ]);
984
985     int load_gmem_b_k = bk * BK + load_smem_b_k;
986     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
987     HALF2(s_b[load_smem_b_k][load_smem_b_n]) = HALF2(b[load_gmem_b_addr
988     ]);
989     __syncthreads();
990
991 #pragma unroll
992     for (int k = 0; k < BK; k++) {
993 #pragma unroll
994         for (int m = 0; m < TM; m++) {
995 #pragma unroll
996             for (int n = 0; n < TN; n++) {
997                 int comp_smem_a_m = ty * TM + m;
998                 int comp_smem_b_n = tx * TN + n;
999                 r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1000             }
1001         }
1002     }
1003     __syncthreads();
1004 }
1005
1006 #pragma unroll
1007 for (int m = 0; m < TM; ++m) {
1008     int store_gmem_c_m = by * BM + ty * TM + m;
1009 #pragma unroll
1010     for (int n = 0; n < TN; n += 2) {
1011         int store_gmem_c_n = bx * BN + tx * TN + n;
1012         int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1013         HALF2(c[store_gmem_c_addr]) = HALF2(r_c[m][n]);
1014     }
1015 }
1016
1017 // =====
1018 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1019 // =====
1020 // 面试要点:
1021 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1022 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
1023 // - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit 寄存器)
1024 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1025 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1026 //
1027 // TN 布局详解 (面试高频考点):
1028 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1029 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1030 // N = Normal (列优先, BLAS 原生格式)
1031 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1032 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1033 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对 BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1034 // 视角下: TN = A row-major [M×K], B col-major [N×K] 在 cuBLAS
1035 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1036 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1037 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1038 //
1039 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1040 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1041 // N = col-major (列优先, BLAS native = normal)
1042 //
1043 // NN 布局对比: C[M×N] = A[M×K] × B[K×N]
1044 // - A: row-major [M, K], B: row-major [K, N]
1045 // - 两者都是行优先 → BLAS 视角都是 T → cuBLAS: (T, T)
1046 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1047
1048 //
1049 // TN 布局: C[M×N] = A[M×K] × B[N×K] (A 行优先, B 列优先)
1050 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1051 // - B [N×K]: col-major → 全局索引 B[n*K + k] (内维是 K!)
1052 // 物理含义: 存的是原 B[K×N] 的转置 B^T
1053 // - 优势: B 已是 col-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1054 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1055 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1056 //
1057 // WGMMA (Warp Group MMA, Hopper+):
1058 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1059 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (比 MMA 大 32 倍)
1060 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1061 // threads) 做计算
1062 // - TMA: 硬件 DMA, 零寄存器开销, 支持 2D/3D 寻址
1063
1064 // =====
1065 // Phase 5b-1: MMA PTX 宏定义
1066 // =====
1067
1068 // ---- gmem → smem: cp.async ----
1069 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n"
1070 :)
1071 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1072 #define CP_ASYNC_WAIT_GROUP(n)
1073     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1074
1075 // cp.async.cg: bypass L1, 写入 L2 (适合 GEMM 中只读一次的数据)
1076 // cp.async.ca: cache all, L1+L2 (适合需要多次复用的数据)
1077 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1078 #define CP_ASYNC_CG(dst, src, bytes)
1079     asm volatile(
1080         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst),
1081         "l"(src), "n"(bytes))
1082
1083 // ---- ldmatrix: smem → register (Tensor Core 专用)----
1084 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1085 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1086 // aligned: 要求 128-bit 对齐
1087 // trans: 转置加载 (用于 col-major 的 B 矩阵)
1088 #define LDMATRIX_X4(R0, R1, R2, R3, addr)
1089     asm volatile(
1090         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n"
1091         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)
1092         : "r"(addr))
1093
1094 #define LDMATRIX_X2(R0, R1, addr)
1095     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n"
1096         : "=r"(R0), "=r"(R1)
1097         : "r"(addr))
1098
1099 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1100 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用
1101 // trans
1102 // 加载
1103 #define LDMATRIX_X2_T(R0, R1, addr)
1104     asm volatile(
1105         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"
1106         : "=r"(R0), "=r"(R1)
1107         : "r"(addr))
1108
1109 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16 ----
1110 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1111 // row.col: A 是 row-major, B 是 col-major
1112 // f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1113 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1114 // C 矩阵大小 16×8=128 元素 = 64 个 half = 2 个 uint32
1115 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)
1116     asm volatile(
1117         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16 {%0, %1}, {%2,
1118         %3, "
1119         "%4, %5}, {%6, %7}, {%8, %9};\n"

```



```

1117         : "=r"(RDO), "=r"(RD1)
1118         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RBO), "r"(RB1), "r"(RC0),
1119         "r"(RC1))
1120
1121 // ---- MMA 辅助函数 ----
1122 // div_ceil: 整数除法向上取整
1123 #define HOST_DEVICE_INLINE __device__ __host__ inline
1124 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1125     return (a % b != 0) ? (a / b + 1) : (a / b);
1126 }
1127
1128 // =====
1129 // Phase 5b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局
1130 // =====
1131 // 面试重点 — Tile Hierarchy:
1132 // MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1133 // MMA Tile (warp):    2x4=8 个 MMA atom = [32x32] (一个 warp 的计算)
1134 // Warp Tile:          4x4=16 warps = [128x128] (整个 block 的计算)
1135 // 线程映射:           2x4=8 warps, 每个 warp 4x1=4 个 warp tile
1136 // 实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
1137 // → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1138 //
1139 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1140 // - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1141 // - B[N][K]: col-major, 全局索引 B[n*K + k] ( 注意内维是 K 不是 N!)
1142 // shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1143 // - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1144 // - ldmatrix B: 用 x2 (非转置), 因为 B 已是 col-major (BLAS Normal), 无需
1145 // .trans
1146 // - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1147 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block
1148 // swizzle
1149 // - grid.z = S 个 swizzle 分区
1150 // - grid.x = 每个分区内部需要发射多少个 128x128 的 N tiles
1151 // - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N
1152 // 区域, 改善
1153 // L2 命中
1154 // Block: (256, 1, 1), 8 warps
1155 // source: LeetCode/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1156 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K =
1157     16,
1158     const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1159     const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1160     const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1161 __global__ void __launch_bounds__(256)
1162     hgemm_mma_m16n8k16_mma2x4_warp4x4_stages_dsmem_tn_kernel(half *A,
1163     half *B,
1164     half *C, int M,
1165     int N, int K) {
1166     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1167     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx
1168     .x;
1169     const int by = blockIdx.y;
1170     constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1171     constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1172     constexpr int BK = MMA_K; // 16
1173     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1174     // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B
1175     // col-major)
1176     // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会
1177     // 额外
1178     // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里
1179     // 先保留最简
1180     // PAD=0 版本。
1181     extern __shared__ half smem[];
1182     half *s_a = smem;
1183     half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1184     constexpr int s_a_stage_offset = BM * BK; // 128*16
1185     constexpr int s_b_stage_offset =
1186         BN * BK; // 128*16 BN(128)×BK(16) = B col-major 的 smem 布局
1187
1188     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1189     const int warp_id = tid / WARP_SIZE; // 0-7
1190     const int lane_id = tid % WARP_SIZE; // 0-31
1191     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1192     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1193
1194     // 线程到 global memory 的映射 (用于加载 A 和 B)
1195     // TN 布局关键: A[m*K+k] 是 row-major, B[n*K+k] 是 col-major (内维是
1196     K!)
1197     int load_smem_a_m = tid / 2; // 0-127
1198     int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1199     int load_smem_b_n = tid / 2; // 0-127 → B 的 N 方向 (col-major 的行)
1200     int load_smem_b_k =
1201         (tid % 2 == 0) ? 0 : 8; // 0, 8 → B 的 K 方向 (col-major 的列)
1202     int load_gmem_a_m = by * BM + load_smem_a_m;
1203     int load_gmem_b_n =
1204         bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏
1205         移
1206     if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1207         return;
1208
1209     // 累加器: 每个 warp 计算 WARP_TILE_M×WARP_TILE_N=16 个 MMA tile
1210     uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1211
1212     // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1213     uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1214     uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1215
1216     // 预加载前 (K_STAGE-1) 个 stage
1217     // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B 用 n*K+k(col-major)
1218     #pragma unroll
1219     for (int k = 0; k < (K_STAGE - 1); ++k) {
1220         int load_gmem_a_k = k * BK + load_smem_a_k;
1221         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k
1222         ]
1223         int load_gmem_b_k = k * BK + load_smem_b_k;
1224         int load_gmem_b_addr =
1225             load_gmem_b_n * K + load_gmem_b_k; // B: [n][k] col-major
1226
1227         uint32_t load_smem_a_ptr =
1228             (smem_a_base_ptr +
1229             (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
1230             sizeof(half));
1231         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1232
1233         uint32_t load_smem_b_ptr =
1234             (smem_b_base_ptr +
1235             (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
1236             sizeof(half));
1237         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1238
1239         CP_ASYNC_COMMIT_GROUP();
1240     }
1241
1242     const int NUM_K_TILES = div_ceil(K, BK);
1243     CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1244     __syncthreads();
1245
1246     // 主循环: K 维分块迭代
1247     #pragma unroll
1248     for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1249         // Stage 选择: 轮转方式 (round-robin)
1250         // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1251         // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
1252         int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1253         int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1254
1255         // 异步加载下一批数据到 smem_sel_next
1256         // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B 的 gmem 地址用
1257         // n*K+k (col-major, 内维是 K)
1258         int load_gmem_a_k = k * BK + load_smem_a_k;
1259         int load_gmem_a_addr =
1260             load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1261         int load_gmem_b_k = k * BK + load_smem_b_k;

```

```

1254     int load_gmem_b_addr =
1255         load_gmem_b_n * K + load_gmem_b_k; // B: col-major [n][k] 内维是K!
1256
1257     uint32_t load_smem_a_ptr =
1258         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1259             load_smem_a_m * BK + load_smem_a_k) *
1260             sizeof(half));
1261     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1262
1263     uint32_t load_smem_b_ptr =
1264         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1265             load_smem_b_n * BK + load_smem_b_k) *
1266             sizeof(half));
1267     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1268     CP_ASYNC_COMMIT_GROUP();
1269
1270     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1271     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1272     // B 用 x2 (非转置), 因为 B 已经是 col-major, 无需 .trans
1273     uint32_t RA[WARP_TILE_M][4];
1274     uint32_t RB[WARP_TILE_N][2];
1275
1276     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1277 #pragma unroll
1278     for (int i = 0; i < WARP_TILE_M; ++i) {
1279         int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1280         int lane_smem_a_m =
1281             warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1282         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1283         uint32_t lane_smem_a_ptr =
1284             (smem_a_base_ptr +
1285                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1286                     lane_smem_a_k) *
1287                     sizeof(half));
1288         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1289     }
1290
1291     // ldmatrix.x2: 加载 B 的 k16n8 片段 (col-major B, 非转置)
1292     // 为什么不用 .trans? 因为 TN 布局下 B 已经是 col-major [N][K]
1293     // ldmatrix 默认期望 col-major 输入 → 直接匹配
1294 #pragma unroll
1295     for (int j = 0; j < WARP_TILE_N; ++j) {
1296         int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1297         int lane_smem_b_n =
1298             warp_smem_b_n + lane_id % 8; // ldmatrix B 用 8 个 lane
1299         int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1300         uint32_t lane_smem_b_ptr =
1301             (smem_b_base_ptr +
1302                 (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1303                     lane_smem_b_k) *
1304                     sizeof(half));
1305         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1306     }
1307
1308     // MMA compute: 发射 WARP_TILE_M * WARP_TILE_N 条 MMA 指令
1309 #pragma unroll
1310     for (int i = 0; i < WARP_TILE_M; ++i) {
1311         for (int j = 0; j < WARP_TILE_N; ++j) {
1312             HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1313                 RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1314         }
1315     }
1316
1317     // 等待当前 stage 的异步加载完成, 然后 sync
1318     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1319     __syncthreads();
1320 }
1321
1322 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1323 if ((K_STAGE - 2) > 0) {
1324     CP_ASYNC_WAIT_GROUP(0);
1325     __syncthreads();
1326 }
1327 {
1328 #pragma unroll
1329     for (int k = 0; k < (K_STAGE - 1); k++) {
1330         uint32_t RA[WARP_TILE_M][4];
1331         uint32_t RB[WARP_TILE_N][2];
1332         int stage_sel = (NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE;
1333
1334 #pragma unroll
1335         for (int i = 0; i < WARP_TILE_M; ++i) {
1336             int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1337             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1338             int lane_smem_a_k = (lane_id / 16) * 8;
1339             uint32_t lane_smem_a_ptr =
1340                 (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1341                     lane_smem_a_m * BK + lane_smem_a_k) *
1342                     sizeof(half));
1343             LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3],
1344                 lane_smem_a_ptr);
1345         }
1346
1347 #pragma unroll
1348         for (int j = 0; j < WARP_TILE_N; ++j) {
1349             int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1350             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1351             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1352             uint32_t lane_smem_b_ptr =
1353                 (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1354                     lane_smem_b_n * BK + lane_smem_b_k) *
1355                     sizeof(half));
1356             LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1357         }
1358
1359 #pragma unroll
1360         for (int i = 0; i < WARP_TILE_M; ++i) {
1361             for (int j = 0; j < WARP_TILE_N; ++j) {
1362                 HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1363                     RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1364             }
1365         }
1366     }
1367
1368     // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1369 {
1370     for (int i = 0; i < WARP_TILE_M; ++i) {
1371         uint32_t RCO[WARP_TILE_N][4];
1372         uint32_t RC1[WARP_TILE_N][4];
1373         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1374         // 对 m16n8k16 的 C fragment:
1375         // - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1376         // - lane+1 持有同一行里的 {c2,c3}
1377         // - lane+2 持有同一行里的 {c4,c5}
1378         // - lane+3 持有同一行里的 {c6,c7}
1379         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由
1380         // lane%4==0 的那个线程做一次 128-bit store。
1381 #pragma unroll
1382         for (int j = 0; j < WARP_TILE_N; ++j) {
1383             RCO[j][0] = RC[i][j][0];
1384             RC1[j][0] = RC[i][j][1];
1385             RCO[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1386             RCO[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1387             RCO[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1388             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1389             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1390             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1391         }
1392
1393         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1394         if (lane_id % 4 == 0) {
1395             int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i *

```

```

MMA_M; // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1394     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id; // m64n128k16: M=64, N=128, K=16
    / 4; // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1395 #pragma unroll // 32 个输出寄存器 = 64x128/2/4 = 1024 个 half / 32 = 32 个 uint32
1396     for (int j = 0; j < WARP_TILE_N; ++j) { // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1397         int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j; // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
MMA_N; // #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB,
1398     int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; TransA,
1399     int store_gmem_c_addr_0 = TransB)
1400     store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1401     int store_gmem_c_addr_1 =
1402     (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1403     // 128-bit store: 一次写入 8 个 half
1404     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1405     *reinterpret_cast<float4*>(&RC0[j][0]);
1406     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1407     *reinterpret_cast<float4*>(&RC1[j][0]);
1408 }
1409 }
1410 }
1411 }
1412 }
1413 }
1414 // =====
1415 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (
Hopper)
1416 // =====
1417 // 面试要点 (WGMMMA vs MMA 对比):
1418 // - MMA: warp 级 (32 threads), 同步执行
1419 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-
forget)
1420 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64x128x16=131K 个乘法 (MMA
的 32
1421 // 倍)
1422 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1423 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过
barrier
1424 // 同步
1425 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1426
1427 // ---- WGMMMA 辅助函数 ----
1428 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "
memory")
1429 #define WGMMMA_COMMIT_GROUP()
1430     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1431 #define WGMMMA_WAIT_GROUP(n)
1432     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :::"n"(n) : "memory")
1433
1434 // Shared memory descriptor encode: 将 smem 地址编码为 WGMMMA 可用的描述符
1435 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1436
1437 // make_smem_desc: 创建 WGMMMA 的 shared memory 矩阵描述符
1438 // 这里沿用原始 hgemm_wgmma 实现里的描述符字段约定:
1439 // - base addr: 当前 shared memory tile 基址
1440 // - leading offset: 16 bytes = 8 个 half * 2B, 对应 WGMMMA 在 tile 内沿
minor
1441 // 方向前进一次的 byte 步长
1442 // - stride offset: 1024 bytes = 64 * 8 * 2B, 对应跨到下一条 major stripe
的 byte 步长
1443 // - bit 62: 打开 128B swizzle
1444 // 这些字段是当前 WGMMMA/TMA 布局下的实现常量, 不要把它直接背成 BM/BK
1445 // 的原始字节数公式。
1446 __device__ inline uint64_t make_smem_desc(half *ptr) {
1447     uint32_t addr = static_cast<uint32_t>((__cvta_generic_to_shared(ptr)));
1448     uint64_t desc = 0x0000000000000000;
1449     desc |= SMEM_DESC_ENCODE(addr);
1450     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16; // leading dim offset
bytes
1451     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32; // stride dim offset
bytes
1452     desc |= 1llu << 62; // 128B swizzle
1453     return desc;
1454 }
1455
1456 // ---- WGMMMA PTX 指令宏 ----

```

```

1517         const CUTensorMap *__restrict__ tensorMapB) {
1518
1519         // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1520         // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1521         // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1522         // 不展开宿主侧 create_tensor_map 细节。
1523
1524         const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx
            .x;
1525         const int by = blockIdx.y;
1526         constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1527         constexpr int B_WG_M = BM / num_consumers;             // 128
1528
1529         if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1530             return;
1531
1532         extern __shared__ __align__(128) uint8_t smem[];
1533         WmmaSMem<BM, BN, BK, K_STAGE> &s =
1534             *reinterpret_cast<WmmaSMem<BM, BN, BK, K_STAGE>*>(smem);
1535         half *s_a = s.A;
1536         half *s_b = s.B;
1537
1538         // CTA barrier: 同步 Producer 和 Consumer
1539         __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1540         __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1541
1542         const int num_blocks_k = K / BK;
1543         const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1544         const int tid = threadIdx.x % 128;    // 0-127 within warpgroup
1545
1546         // 初始化 barriers (仅 thread 0 执行)
1547         if (threadIdx.x == 0) {
1548             for (int i = 0; i < K_STAGE; ++i) {
1549                 // 这里的 128 不是“warp 数”也不是“线程块总线程数”,
1550                 // 而是这个 barrier 上每轮会 arrive 的参与者总数: 128 个 consumer 线
1551                 // 程 + 1 个 producer 提交线程。
1552                 init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1
1553                 producer
1554                 init(&empty[i], num_consumers * 128 + 1); // same
1555             }
1556             cuda::device::experimental::fence_proxy_async_shared_cta();
1557         }
1558         __syncthreads();
1559         // ===== Producer Warpgroup (WG0) =====
1560         if (wg_idx == 0) {
1561             if (tid == 0) { // 仅 producer 的 thread 0 执行 TMA 加载
1562                 // TMA 是硬件 DMA 提交指令: 发起一次 descriptor+坐标提交后,
1563                 // 后续搬运由硬件异步完成,
1564                 // 不需要整个 producer warpgroup 里的 128 个线程都参与拷贝。
1565                 int qidx = 0;
1566                 for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1567                     ++block_k_iter, ++qidx) {
1568                     if (qidx == K_STAGE)
1569                         qidx = 0;
1570
1571                     // 等待 Consumer 释放此 stage (empty 信号)
1572                     empty[qidx].wait(empty[qidx].arrive());
1573
1574                     // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1575                     cuda::device::experimental::
1576                     cp_async_bulk_tensor_2d_global_to_shared(
1577                         &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1578                         full[qidx]);
1579
1580                     // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1581                     cuda::device::experimental::
1582                     cp_async_bulk_tensor_2d_global_to_shared(
1583                         &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1584                         full[qidx]);
1585
1586                     // 通知 Consumer 此 stage 已准备好 (full 信号)
1587                     cuda::device::barrier_arrive_tx(full[qidx], 1,

```



```

1656 #pragma unroll
1657 for (int g = 0; g < WGMMMA_N / 16; ++g) {
1658     int col = g * 16 + 2 * (lane % 4);
1659     // 将 uint32 直接 reinterpret 为 half2 pair 写入 C
1660     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1661         d[m_it][g][0];
1662     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1663         d[m_it][g][1];
1664     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1665         d[m_it][g][2];
1666     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1667         d[m_it][g][3];
1668 }
1669 }
1670 }
1671 }
1672
1673 // =====
1674 // Phase 6: RoPE — 旋转位置编码 (Rotary Position Embedding)
1675 // =====
1676 // 面试要点:
1677 // - RoPE 数学公式: 对每对相邻维度做 2D 旋转
1678 // [x1']   [cos()  -sin()] [x1]
1679 // [x2'] = [sin()   cos()] [x2]
1680 // - _i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
1681 // - token_pos = idx / N: token 在序列中的位置
1682 // - token_idx = idx % N: token 内的维度对索引
1683 // - 输入 [seq_len, hidden_size], 输出同形状
1684
1685 // Grid: ((seq_len * N + 255) / 256, 1, 1)
1686 // Block: (256, 1, 1)
1687 // source: LeetCUDA/kernels/rope/rope.cu
1688 __global__ void rope_f32_kernel(float *x, float *out, int seq_len, int N)
1689 {
1690     int idx = blockIdx.x * blockDim.x + threadIdx.x;
1691     float x1 = x[idx * 2];
1692     float x2 = x[idx * 2 + 1];
1693     int token_pos = idx / N; // 序列位置
1694     int token_idx = idx % N; // 维度对索引
1695
1696     // 频率计算: _i = 1 / (10000^(2i/d))
1697     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
1698     float sin_v = sinf(token_pos * exp_v);
1699     float cos_v = cosf(token_pos * exp_v);
1700
1701     // 2D 旋转
1702     float out1 = x1 * cos_v - x2 * sin_v;
1703     float out2 = x1 * sin_v + x2 * cos_v;
1704     out[idx * 2] = out1;
1705     out[idx * 2 + 1] = out2;
1706 }
1707 // =====
1708 // Phase 7: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
1709 // =====
1710 // 面试要点 (Bank Conflict 专题):
1711 // - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
1712 // - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
1713 // - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
1714 // - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
1715 //
1716 // 转置的三步演进:
1717 // naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
1718 // shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
1719 // BCF: smem 布局 [WARP_SIZE_S][WARP_SIZE_S*4+PAD], PAD=1 消除 bank
1720 // conflict
1721 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
1722
1723 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写)----
1724 // 每个线程处理 1 个元素, block(16,16)
1725 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取
1726 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 (32 次内存事务)
1727
1728 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
1729 // Block: (16, 16, 1)
1730 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1731 __global__ void mat_transpose_f32_row2col2d_kernel(float *x, float *y,
1732     const int row,
1733     const int col) {
1734     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1735     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1736     if (global_y < row && global_x < col) {
1737         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
1738         // 写入: y[global_y][global_x] = y[global_y * row + global_x], 列优先, 非合并
1739         y[global_y * row + global_x] = x[global_x * col + global_y];
1740     }
1741 }
1742 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write)
1743 // ----
1744 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
1745 // Block: (16, 16, 1)
1746 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
1747 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1748 __global__ void
1749     mat_transpose_f32x4_shared_bcf_merge_write_row2col2d_kernel(
1750     float *x, float *y, const int row, const int col) {
1751     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1752     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1753     const int local_x = threadIdx.x;
1754     const int local_y = threadIdx.y;
1755
1756     constexpr int WARP_SIZE_S = 16;
1757     constexpr int PAD = 1;
1758     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
1759     __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
1760
1761     if (global_y * 4 < row && global_x < col) {
1762         // Step 1: 从 global memory 读取 4 行, 写入 shared
1763         // memory (行优先, 合并读取)
1764         float4 x_val;
1765         x_val.x = x[(global_y * 4) * col + global_x];
1766         x_val.y = x[(global_y * 4 + 1) * col + global_x];
1767         x_val.z = x[(global_y * 4 + 2) * col + global_x];
1768         x_val.w = x[(global_y * 4 + 3) * col + global_x];
1769         tile[local_y * 4][local_x] = x_val.x;
1770         tile[local_y * 4 + 1][local_x] = x_val.y;
1771         tile[local_y * 4 + 2][local_x] = x_val.z;
1772         tile[local_y * 4 + 3][local_x] = x_val.w;
1773         __syncthreads();
1774
1775         // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
1776         float4 smem_val;
1777         smem_val.x = tile[local_x * 4][local_y];
1778         smem_val.y = tile[local_x * 4 + 1][local_y];
1779         smem_val.z = tile[local_x * 4 + 2][local_y];
1780         smem_val.w = tile[local_x * 4 + 3][local_y];
1781
1782         const int gid_x = blockIdx.x * blockDim.x;
1783         const int gid_y = blockIdx.y * blockDim.y * 4;
1784         const int out_y = gid_y + local_x * 4;
1785         const int out_x = gid_x + local_y;
1786
1787         // float4 merge write: 1 次 128-bit store; 这里也隐含 out_y 为 4 的倍数
1788         *reinterpret_cast<float4*>(&y[(out_x * row + out_y) / 4]) =
1789             *reinterpret_cast<float4*>(&smem_val);
1790     }
1791 }
1792 // =====
1793 // Phase 8: 杂项 Ops — Dot Product / Block All Reduce / Histogram
1794 // =====
1795 // 面试要点:
1796 // - Dot Product: elementwise mul + reduce, 演示两种 reduce 模式结合

```

```

1797 // - Block All Reduce: 多 block 各自做 reduce, 最后 atomicAdd 到全局结果
1798 // - Histogram: atomicAdd 模式, 演示共享内存冲突的处理
1799
1800 // ---- Dot Product: y = sum(a[i] * b[i]) ----
1801 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
1802 template <const int NUM_THREADS = 128>
1803 // Grid: ((N + 127) / 128, 1, 1)
1804 // Block: (128, 1, 1)
1805 // source: LeetCUDA/kernels/dot-product/dot_product.cu
1806 __global__ void dot(float *a, float *b, float *y, int N) {
1807     int tid = threadIdx.x;
1808     int idx = blockIdx.x * NUM_THREADS + tid;
1809     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1810     __shared__ float reduce_smem[NUM_WARPS];
1811
1812     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
1813     int warp = tid / WARP_SIZE;
1814     int lane = tid % WARP_SIZE;
1815
1816     prod = warp_reduce_sum<WARP_SIZE>(prod);
1817     if (lane == 0)
1818         reduce_smem[warp] = prod;
1819     __syncthreads();
1820
1821     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1822     if (warp == 0)
1823         prod = warp_reduce_sum<NUM_WARPS>(prod);
1824     if (tid == 0)
1825         atomicAdd(y, prod);
1826 }
1827
1828 // Dot Product + float4
1829 template <const int NUM_THREADS = 128 / 4>
1830 // Grid: ((N + 127) / 128, 1, 1)
1831 // Block: (32, 1, 1), 128/4=32
1832 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
1833 // source: LeetCUDA/kernels/dot-product/dot_product.cu
1834 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
1835     int tid = threadIdx.x;
1836     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
1837     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1838     __shared__ float reduce_smem[NUM_WARPS];
1839
1840     float4 reg_a = FLOAT4(a[idx]);
1841     float4 reg_b = FLOAT4(b[idx]);
1842     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
1843                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
1844                             : 0.0f;
1845     int warp = tid / WARP_SIZE;
1846     int lane = tid % WARP_SIZE;
1847
1848     prod = warp_reduce_sum<WARP_SIZE>(prod);
1849     if (lane == 0)
1850         reduce_smem[warp] = prod;
1851     __syncthreads();
1852
1853     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1854     if (warp == 0)
1855         prod = warp_reduce_sum<NUM_WARPS>(prod);
1856     if (tid == 0)
1857         atomicAdd(y, prod);
1858 }
1859
1860 // ---- Block All Reduce Sum: y = sum(a[0..N-1]) ----
1861 // 多 block 各自做 warp+smem→warp0 reduce, 然后 atomicAdd 到全局 y
1862 template <const int NUM_THREADS = 128>
1863 // Grid: ((N + 127) / 128, 1, 1)
1864 // Block: (128, 1, 1)
1865 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
1866 __global__ void block_all_reduce_sum(float *a, float *y, int N) {
1867     int tid = threadIdx.x;
1868     int idx = blockIdx.x * NUM_THREADS + tid;
1869     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1870     __shared__ float reduce_smem[NUM_WARPS];
1871
1872     float sum = (idx < N) ? a[idx] : 0.0f;
1873     int warp = tid / WARP_SIZE;
1874     int lane = tid % WARP_SIZE;
1875
1876     sum = warp_reduce_sum<WARP_SIZE>(sum);
1877     if (lane == 0)
1878         reduce_smem[warp] = sum;
1879     __syncthreads();
1880
1881     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1882     if (warp == 0)
1883         sum = warp_reduce_sum<NUM_WARPS>(sum);
1884     if (tid == 0)
1885         atomicAdd(y, sum);
1886 }
1887
1888 // Block All Reduce Sum + float4
1889 template <const int NUM_THREADS = 128 / 4>
1890 // Grid: ((N + 127) / 128, 1, 1)
1891 // Block: (32, 1, 1), 128/4=32
1892 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
1893 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
1894 __global__ void block_all_reduce_sum_vec4(float *a, float *y, int N) {
1895     int tid = threadIdx.x;
1896     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
1897     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1898     __shared__ float reduce_smem[NUM_WARPS];
1899
1900     float4 reg_a = FLOAT4(a[idx]);
1901     float sum = (idx < N) ? (reg_a.x + reg_a.y + reg_a.z + reg_a.w) : 0.0f;
1902     int warp = tid / WARP_SIZE;
1903     int lane = tid % WARP_SIZE;
1904
1905     sum = warp_reduce_sum<WARP_SIZE>(sum);
1906     if (lane == 0)
1907         reduce_smem[warp] = sum;
1908     __syncthreads();
1909
1910     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1911     if (warp == 0)
1912         sum = warp_reduce_sum<NUM_WARPS>(sum);
1913     if (tid == 0)
1914         atomicAdd(y, sum);
1915 }
1916
1917 // ---- Histogram: y[a[i]]++ ----
1918 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
1919 // Grid: ((N + 255) / 256, 1, 1)
1920 // Block: (256, 1, 1)
1921 // source: LeetCUDA/kernels/histogram/histogram.cu
1922 __global__ void histogram(int *a, int *y, int N) {
1923     int idx = blockIdx.x * blockDim.x + threadIdx.x;
1924     if (idx < N)
1925         atomicAdd(&y[a[idx]], 1);
1926 }
1927
1928 // Histogram + int4 向量化
1929 // Grid: ((N + 255) / 256, 1, 1)
1930 // Block: (64, 1, 1), 256/4=64
1931 // 注意: 该版本默认输入地址满足 int4 对齐; 最适合 N 按 4 对齐的场景
1932 // source: LeetCUDA/kernels/histogram/histogram.cu
1933 __global__ void histogram_vec4(int *a, int *y, int N) {
1934     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
1935     if (idx < N) {
1936         int4 reg_a = INT4(a[idx]);
1937         atomicAdd(&y[reg_a.x], 1);
1938         atomicAdd(&y[reg_a.y], 1);
1939         atomicAdd(&y[reg_a.z], 1);
1940         atomicAdd(&y[reg_a.w], 1);
1941     }
1942 }
1943
1944 // =====
1945 // Phase 9: FlashAttention-2 (Split-Q + MMA m16n8k16)
1946 // =====

```

```
1947 // 面试要点 (FlashAttention 算法):
1948 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
1949 // 但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
1950 // 2. FA 三板斧:
1951 // a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
1952 // b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
1953 // c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
1954 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
1955 // - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
1956 // - 优点: 减少 warp 间通信和 shuffle
1957 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
1958 // for each K,V tile:
1959 //   S_cur = Q @ K^T * scale
1960 //   m_new = max(m_old, row_max(S_cur))
1961 //   P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
1962 //   l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
1963 //   O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
1964 //   O_final = O_new / l_final
1965 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
1966 // - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
1967 // - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
1968 // 后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
1969 // - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
1970 // 都天然同构”的通用结论
1971 //
1972 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
1973 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
1974 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
1975 // Block: (128, 1, 1), kNumThreads=
1976 //   WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
1977 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.
1978 //   cu
1979 // ---- 寄存器填充辅助函数 ----
1980 template <typename T, int M, const int N, const int K = 2>
1981 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
1982     for (int i = 0; i < M; ++i)
1983         for (int j = 0; j < N; ++j)
1984             for (int k = 0; k < K; ++k)
1985                 R[i][j][k] = val;
1986 }
1987
1988 //
1989 //
1990 template <typename T, int M, const int N = 2>
1991 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
1992     for (int i = 0; i < M; ++i)
1993         for (int j = 0; j < N; ++j)
1994             R[i][j] = val;
1995 }
1996 //
1997 //
1998 //
1999 // =====
2000 // FlashAttention-2 Split-Q Kernel (完整实现)
2001 // =====
2002 // Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2003 //
2004 // Tile 设计 (以 kHeadDim=64 为例):
2005 // Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ = 16*4*1 = 64
2006 // Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK = 8*1*8 = 64
2007 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2008 //
2009 // 执行流程:
2010 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2011 // 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2012 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2013 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2014 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale -
2015 //   max)
2016 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2017 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S
2018 //   做 A
2019 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2020 // 7) 3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2021 // 8) 最终 rescale: O_final = (1/l_final) * O_final
2022 // 9) Epilogue: warp shuffle + 128-bit collective store
2023
2024 template <
2025     const int kHeadDim, // head dim: 32, 64, 128
2026     const int kMmaAtomM, // 16 (MMA instruction M dimension)
2027     const int kMmaAtomN, // 8 (MMA instruction N dimension)
2028     const int kMmaAtomK, // 16 (MMA instruction K dimension)
2029     const int kMmaTileSeqLenQ, // MMA tiles along Q's M dim, 4 → Br
2030         =16*4=64
2031     const int kMmaTileSeqLenK, // MMA tiles along K's N dim, 1 → Bc
2032         basis=8
2033     const int kMmaTileSeqLenP, // MMA tiles for P@V M dim, must equal
2034         // kMmaTileSeqLenQ
2035     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim
2036         direction)
2037     const int kWarpTileSeqLenQ, // warp tiles along Q's M, 1 → Br per
2038         warp=16
2039     const int kWarpTileSeqLenK, // warp tiles along K's N, 8 → Bc_warp
2040         =8*8=64
2041     const int kWarpTileSeqLenP, // warp tiles for P@V M dim, 1
2042     const int kWarpTileHeadDimV, // warp tiles for P@V N dim,
2043         // kHeadDim/(8*kMmaTileHeadDimV)
2044     const int kStage, // pipeline stages for K: 1 or 2
2045     const int kPad> // padding for bank conflict avoidance
2046 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ *
2047     kMmaTileSeqLenK)
2048     flash_attn_mma_stages_split_q_kernel(half *Q, half *K, half *V, half
2049     *O,
2050     int QKV_seqlen, int QKV_head) {
2051 // Tile dimensions
2052 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ; //
2053     64
2054 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK; //
2055     64
2056 constexpr int kNumThreads =
2057     WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2058 const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2059 // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处
2060 // 理。
2061 // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理
2062 // 了尾 tile。
2063 const float scale = 1.0f / sqrtf((float)kHeadDim);
2064 // Block indexing
2065 const int QKV_batch_id = blockIdx.y / QKV_head;
2066 const int QKV_head_id = blockIdx.y % QKV_head;
2067 const int Q_tile_id = blockIdx.x;
2068 const int tid = threadIdx.x;
2069 const int warp_id = tid / WARP_SIZE;
2070 const int lane_id = tid % WARP_SIZE;
2071 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2072 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2073 // Global memory base offsets for this (batch, head)
2074 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2075 const int Q_gmem_offset =
2076     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2077     kHeadDim;
2078 const int K_gmem_offset = Q_gmem_offset;
2079 const int V_gmem_offset = Q_gmem_offset;
2080 const int O_gmem_offset = Q_gmem_offset;
2081 // Thread-to-smem mapping for cooperative load
2082 int load_smem_Q_Br = tid / (kNumThreads / Br);
2083 int load_smem_Q_d =
2084     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2085 int load_smem_K_Bc = tid / (kNumThreads / Bc);
2086 int load_smem_K_d =
2087     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2088 int load_smem_V_Bc = tid / (kNumThreads / Bc);
2089 int load_smem_V_d =
2090     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
```

```

2082
2083 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2084 if (load_gmem_Q_Br >= QKV_seqlen)
2085     return;
2086
2087 // ---- Shared memory layout ----
2088 extern __shared__ half smem[];
2089 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+
2090 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc,
2091 half *Q_tile_smem = smem;
2092 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2093 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage
2094 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2095 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2096
2097 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2098 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2099 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2100
2101 // ---- Online Softmax persistent state ----
2102 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2103 // lane_block_row_sum_old[i][r]: running denominator l for row r of
2104 // warp tile
2105 // i
2106 float lane_block_row_max_old[kWarpTileSeqLenQ][2];
2107 float lane_block_row_sum_old[kWarpTileSeqLenQ][2];
2108 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_max_old, -
2109 INFINITY);
2110 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2111
2112 // ---- Register allocation ----
2113 uint32_t R_Q[kWarpTileSeqLenQ][4]; // Q regs
2114 uint32_t R_K[kWarpTileSeqLenK][2]; // K regs
2115 uint32_t R_V[kWarpTileHeadDimV][2]; // V regs
2116 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2117 // 对单个 m16n8k16 tile 而言:
2118 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
2119 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
2120 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器
2121 // 内变换。
2122 uint32_t R_S[kWarpTileSeqLenQ][kWarpTileSeqLenK][2]; // S=Q@K^T / P=
2123 softmax(S)
2124 uint32_t R_O[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // O for current
2125 tile
2126 uint32_t R_D[kWarpTileSeqLenP][kWarpTileHeadDimV]
2127 [2]; // O accumulator (final output)
2128
2129 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
2130 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_D, 0);
2131
2132 // =====
2133 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2134 // =====
2135 {
2136     int load_gmem_Q_addr =
2137         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2138     uint32_t load_smem_Q_ptr =
2139         smem_Q_base_ptr +
2140         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(
2141             half);
2142 #pragma unroll
2143     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2144         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2145     }
2146     CP_ASYNC_COMMIT_GROUP();
2147 }
2148
2149 // =====
2150 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2151 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从
2152 // tile 0
2153 // 开始,
2147 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2148 // =====
2149 if constexpr (kStage > 1) {
2150 #pragma unroll
2151     for (int stage = 0; stage < (kStage - 1); ++stage) {
2152         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2153         int load_gmem_K_addr =
2154             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2155         uint32_t load_smem_K_ptr =
2156             smem_K_base_ptr +
2157             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2158             load_smem_K_d) *
2159             sizeof(half);
2160 #pragma unroll
2161         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2162             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i],
2163             16);
2164         }
2165         CP_ASYNC_COMMIT_GROUP();
2166     }
2167     CP_ASYNC_WAIT_GROUP(kStage - 2);
2168     __syncthreads();
2169 }
2170
2171 // =====
2172 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2173 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2174 // =====
2175 #pragma unroll 1
2176 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2177     int smem_sel = tile_K_seqlen % kStage;
2178     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2179
2180 // ---- 3a: 异步加载 K/V tile (多 stage pipeline)----
2181 if constexpr (kStage > 1) {
2182     // 只有 kStage>1 才能真正做 K 的 pipeline:
2183     // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮
2184     // 预取” 的 K
2185     // tile。
2186     // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖
2187     // 同一块
2188     // smem。
2189     // Load current V tile (no pipeline for V — one stage is enough)
2190     {
2191         int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2192         int load_gmem_V_addr =
2193             V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2194         uint32_t load_smem_V_ptr =
2195             smem_V_base_ptr +
2196             (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof
2197             (half);
2198 #pragma unroll
2199         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2200             CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i],
2201             16);
2202         }
2203         CP_ASYNC_COMMIT_GROUP();
2204     }
2205
2206 // Prefetch next K tile (pipelined)
2207 if ((tile_K_seqlen + 1) < Tc) {
2208     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2209     int load_gmem_K_addr =
2210         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2211     uint32_t load_smem_K_ptr =
2212         smem_K_base_ptr +
2213         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim +
2214         kPad) +
2215         load_smem_K_d) *
2216         sizeof(half);
2217 #pragma unroll
2218     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2219         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i],
2220         16);
2221     }

```



```

2215     CP_ASYNC_COMMIT_GROUP();
2216 }
2217 }
2218
2219 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
2220 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S,
0);
2221 #pragma unroll
2222 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d)
2223 {
2224     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2225 #pragma unroll
2226 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2227     int warp_smem_Q_Br =
2228         warp_QP * (kMmaAtomM * kWarpTileSeqLenQ) + i * kMmaAtomM;
2229     int lane_smem_Q_Br =
2230         warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2231     int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; //
0, 8
2232     uint32_t lane_smem_Q_ptr =
2233         smem_Q_base_ptr +
2234         (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof
(half);
2235     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2236         lane_smem_Q_ptr);
2237 }
2238 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2239 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2240 #pragma unroll
2241 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2242     int warp_smem_K_Bc =
2243         warp_KV * (kMmaAtomN * kWarpTileSeqLenK) + j * kMmaAtomN;
2244     int lane_smem_K_Bc =
2245         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2246     int lane_smem_K_d =
2247         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2248     uint32_t lane_smem_K_ptr =
2249         smem_K_base_ptr +
2250         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad)
+
2251         lane_smem_K_d) *
2252         sizeof(half);
2253     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2254 }
2255 // MMA: S[tile] += Q[tile] @ K^T[tile]
2256 #pragma unroll
2257 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2258 #pragma unroll
2259 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2260     HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q
[i][2],
2261         R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2262         R_S[i][j][1]);
2263 }
2264 }
2265 } // end loop over d
2266 __syncthreads();
2267 // =====
2268 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P
back to
2269 // R_S
2270 // =====
2271 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2272 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0-7 里的两个 half 值 {c0,
c1}
2273 // - R_S[i][j][1] 对应 rows 8-15 里的两个 half 值 {c2,c3}
2274 // - lane 0-3 持有 row 0 的片段, lane 4-7 持有 row 1, ..., lane 28-31
持有 row 7
2275 // - 对于 rows 8-15 也是同样的 lane 分组, 只是读取的是 reg[1]
2276 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的
是
2277
2278
2279 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2280 // Each (i, j) pair = one 16x8 MMA tile; there are kWarpTileSeqLenQ x
2281 // kWarpTileSeqLenK tiles.
2282
2283 float lane_row_max_new[kWarpTileSeqLenQ][2];
2284 float lane_row_sum_new[kWarpTileSeqLenQ][2];
2285 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_max_new, -INFINITY
);
2286 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2287
2288 // ---- Pass 1: Thread-level reduce max across all columns (
kWarpTileSeqLenK
2289 // tiles) ----
2290 #pragma unroll
2291 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2292 #pragma unroll
2293 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2294     // Extract half values from R_S registers (C matrix fragment
layout)
2295     float2 t_reg_S_0 =
2296         __half22float2(HALF2(R_S[i][j][0])); // rows 0-7: {c0, c1}
2297     float2 t_reg_S_1 =
2298         __half22float2(HALF2(R_S[i][j][1])); // rows 8-15: {c2, c3}
2299     // S = (Q@K^T) * scale
2300     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2301     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2302     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2303     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2304 }
2305 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2306 // {0,4,8,...,28} hold valid data)
2307 lane_row_max_new[i][0] =
2308     warp_reduce_max<float, 4>(lane_row_max_new[i][0]);
2309 lane_row_max_new[i][1] =
2310     warp_reduce_max<float, 4>(lane_row_max_new[i][1]);
2311 }
2312
2313 // ---- Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
----
2314 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2315 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2316 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重
组。
2317 #pragma unroll
2318 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2319     // m_new = max(m_old, m_cur)
2320     float block_row_max_new_0 =
2321         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2322     float block_row_max_new_1 =
2323         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2324
2325 #pragma unroll
2326 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2327     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2328     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2329
2330     // P = exp(S * scale - m_new), 用 fma 保证精度
2331     t_reg_S_0.x =
2332         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2333     t_reg_S_0.y =
2334         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2335     t_reg_S_1.x =
2336         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2337     t_reg_S_1.y =
2338         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2339
2340     // Accumulate row sums
2341     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2342     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2343
2344     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2345     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2346     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2347 }
2348

```

```

2349 // Warp-level reduce sum (warp_size = 4, same as max)
2350 lane_row_sum_new[i][0] =
2351     warp_reduce_sum<float, 4>(lane_row_sum_new[i][0]);
2352 lane_row_sum_new[i][1] =
2353     warp_reduce_sum<float, 4>(lane_row_sum_new[i][1]);
2354 }
2355 __syncthreads();
2356
2357 // =====
2358 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2359 // =====
2360 // Wait for V to be ready before computing P@V
2361 if constexpr (kStage > 1) {
2362     if ((tile_K_seqlen + 1) < Tc) {
2363         CP_ASYNC_WAIT_GROUP(1);
2364     } else {
2365         CP_ASYNC_WAIT_GROUP(0);
2366     }
2367 } else {
2368     CP_ASYNC_WAIT_GROUP(0);
2369 }
2370 __syncthreads();
2371
2372 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_0,
2373     0);
2374
2375 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2376 // Bc=kMmaAtomK=16 + 1 iteration for kHeadDim 64 configurations
2377 #pragma unroll
2378 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2379     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2380     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major
2381     // use
2382     // transposed ldmatrix
2383     #pragma unroll
2384     for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2385         int warp_smem_V_d =
2386             warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2387         int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2388         int lane_smem_V_d = warp_smem_V_d;
2389         uint32_t lane_smem_V_ptr =
2390             smem_V_base_ptr +
2391             (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(
2392                 half);
2393         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2394     }
2395
2396     // P matrix layout in R_S[i][j][2]:
2397     // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2398     // | 64x64 | warp_KV 0 |
2399     // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2400     // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2401     // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2402     // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2403     // tile_V_Bc selects which 16-column slice of P to use:
2404     // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2405     // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2406     // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2407     // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2408     // 对应的 MMA A fragment 布局可以这样记:
2409     // - rows 0-7: lane 0-3 → {a0,a1} 与 {a4,a5}, lane 4-7 → 下一
2410     // 行, 依次类推
2411     // - rows 8-15: lane 0-3 → {a2,a3} 与 {a6,a7}, lane 4-7 → 下一
2412     // 行, 依次类推
2413     // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以
2414     // 直接对接,
2415     // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对
2416     // 所有 MMA
2417     // fragment 都无条件成立的通用结论。
2418     int w = tile_V_Bc * 2;
2419     #pragma unroll
2420     for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2421         #pragma unroll
2422         for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2423             HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023
3024
3025
3026
3027
3028
3029
3030
3031
3032
3033
3034
3035
3036
3037
3038
3039
3040
3041
3042
3043
3044
3045
3046
3047
3048
3049
3050
3051
3052
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128
3129
3130
3131
3132
3133
3134
3135
3136
3137
3138
3139
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161
3162
3163
3164
3165
3166
3167
3168
3169
3170
3171
3172
3173
3174
3175
3176
3177
3178
3179
3180
3181
3182
3183
3184
3185
3186
3187
3188
3189
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
3200
3201
3202
3203
3204
3205
3206
3207
3208
3209
3210
3211
3212
3213
3214
3215
3216
3217
3218
3219
3220
3221
3222
3223
3224
3225
3226
3227
3228
3229
3230
3231
3232
3233
3234
3235
3236
3237
3238
3239
3240
3241
3242
3243
3244
3245
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256
3257
3258
3259
3260
3261
3262
3263
3264
3265
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
3292
3293
3294
3295
3296
3297
3298
3299
3300
3301
3302
3303
3304
3305
3306
3307
3308
3309
3310
3311
3312
3313
3314
3315
3316
3317
3318
3319
3320
3321
3322
3323
3324
3325
3326
3327
3328
3329
3330
3331
3332
3333
3334
3335
3336
3337
3338
3339
3340
3341
3342
3343
3344
3345
3346
3347
3348
3349
3350
3351
3352
3353
3354
3355
3356
3357
3358
3359
3360
3361
3362
3363
3364
3365
3366
3367
3368
3369
3370
3371
3372
3373
3374
3375
3376
3377
3378
3379
3380
3381
3382
3383
3384
3385
3386
3387
3388
3389
3390
3391
3392
3393
3394
3395
3396
3397
3398
3399
3400
3401
3402
3403
3404
3405
3406
3407
3408
3409
3410
3411
3412
3413
3414
3415
3416
3417
3418
3419
3420
3421
3422
3423
3424
3425
3426
3427
3428
3429
3430
3431
3432
3433
3434
3435
3436
3437
3438
3439
3440
3441
3442
3443
3444
3445
3446
3447
3448
3449
3450
3451
3452
3453
3454
3455
3456
3457
3458
3459
3460
3461
3462
3463
3464
3465
3466
3467
3468
3469
3470
3471
3472
3473
3474
3475
3476
3477
3478
3479
3480
3481
3482
3483
3484
3485
3486
3487
3488
3489
3490
3491
3492
3493
3494
3495
3496
3497
3498
3499
3500
3501
3502
3503
3504
3505
3506
3507
3508
3509
3510
3511
3512
3513
3514
3515
3516
3517
3518
3519
3520
3521
3522
3523
3524
3525
3526
3527
3528
3529
3530
3531
3532
3533
3534
3535
3536
3537
3538
3539
3540
3541
3542
3543
3544
3545
3546
3547
3548
3549
3550
3551
3552
3553
3554
3555
3556
3557
3558
3559
3560
3561
3562
3563
3564
3565
3566
3567
3568
3569
3570
3571
3572
3573
3574
3575
3576
3577
3578
3579
3580
3581
3582
3583
3584
3585
3586
3587
3588
3589
3590
3591
3592
3593
3594
3595
3596
3597
3598
3599
3600
3601
3602
3603
3604
3605
3606
3607
3608
3609
3610
3611
3612
3613
3614
3615
3616
3617
3618
3619
3620
3621
3622
3623
3624
3625
3626
3627
3628
3629
3630
3631
3632
3633
3634
3635
3636
3637
3638
3639
3640
3641
3642
3643
3644
3645
3646
3647
3648
3649
3650
3651
3652
3653
3654
3655
3656
3657
3658
3659
3660
3661
3662
3663
3664
3665
3666
3667
3668
3669
3670
3671
3672
3673
3674
3675
3676
3677
3678
3679
3680
3681
3682
3683
3684
3685
3686
3687
3688
3689
3690
3691
3692
3693
3694
3695
3696
3697
3698
3699
3700
3701
3702
3703
3704
3705
3706
3707
3708
3709
3710
3711
3712
3713
3714
3715
3716
3717
3718
3719
3720
3721
3722
3723
3724
3725
3726
3727
3728
3729
3730
3731
3732
3733
3734
3735
3736
3737
3738
3739
3740
3741
3742
3743
3744
3745
3746
3747
3748
3749
3750
3751
3752
3753
3754
3755
3756
3757
3758
3759
3760
3761
3762
3763
3764
3765
3766
3767
3768
3769
3770
3771
3772
3773
3774
3775
3776
3777
3778
3779
3780
3781
3782
3783
3784
3785
3786
3787
3788
3789
3790
3791
3792
3793
3794
3795
3796
3797
3798
3799
3800
3801
3802
3803
3804
3805
3806
3807
3808
3809
3810
3811
3812
3813
3814
3815
3816
3817
3818
3819
3820
3821
3822
3823
3824
3825
3826
3827
3828
3829
3830
3831
3832
3833
3834
3835
3836
3837
3838
3839
3840
3841
3842
3843
3844
3845
3846
3847
3848
3849
3850
3851
3852
3853
3854
3855
3856
3857
3858
3859
3860
3861
3862
3863
3864
3865
3866
3867
3868
3869
3870
3871
3872
3873
3874
3875
3876
3877
3878
3879
3880
3881
3882
3883
3884
3885
3886
3887
3888
3889
3890
3891
3892
3893
3894
3895
3896
3897
3898
3899
3900
3901
3902
3903
3904
3905
3906
3907
3908
3909
3910
3911
3912
3913
3914
3915
3916
3917
3918
3919
3920
3921
3922
3923
3924
3925
3926
3927
3928
3929
3930
3931
3932
3933
3934
3935
3936
3937
3938
3939
3940
3941
3942
3943
3944
3945
3946
3947
3948
3949
3950
3951
3952
3953
3954
3955
3956
3957
3958
3959
3960
3961
3962
3963
3964
3965
3966
3967
3968
3969
3970
3971
3972
3973
3974
3975
3976
3977
3978
3979
3980
3981
3982
3983
3984
3985
3986
3987
3988
3989
3990
3991
3992
3993
3994
3995
3996
3997
3998
3999
4000

```

```

2483
2484 // Update m
2485 lane_block_row_max_old[i][0] = block_row_max_new_0;
2486 lane_block_row_max_old[i][1] = block_row_max_new_1;
2487 }
2488
2489 // Wait for next K tile to be ready in smem before next iteration
2490 if constexpr (kStage > 1) {
2491     if ((tile_K_seqlen + 1) < Tc) {
2492         CP_ASYNC_WAIT_GROUP(0);
2493     }
2494     __syncthreads();
2495 }
2496 } // end outer loop over K seqlen
2497 __syncthreads();
2498
2499 // =====
2500 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
2501 // =====
2502 #pragma unroll
2503 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2504     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2505     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2506 #pragma unroll
2507 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2508     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2509     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2510     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2511     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2512     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2513     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2514     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2515     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2516 }
2517 }
2518
2519 // =====
2520 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit
store
2521 // =====
2522 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0-3,
2523 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2524 #pragma unroll
2525 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2526 #pragma unroll
2527 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2528     uint32_t R_Z[2][4];
2529     R_Z[0][0] = R_D[i][j][0];
2530     R_Z[1][0] = R_D[i][j][1];
2531     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2532     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2533     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2534     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2535     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2536     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2537
2538     // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2539     if (lane_id % 4 == 0) {
2540         int store_warp_regs_0_Br =
2541             warp_QP * (kMmaAtomM * kWarpTileSeqLenP) + i * kMmaAtomM;
2542         int store_lane_gmem_0_Br =
2543             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2544         int store_warp_regs_0_d =
2545             warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2546         int store_lane_gmem_0_d = store_warp_regs_0_d;
2547         int store_gmem_0_addr_0 = 0_gmem_offset +
2548             (store_lane_gmem_0_Br + 0) * kHeadDim +
2549             store_lane_gmem_0_d;
2550         int store_gmem_0_addr_1 = 0_gmem_offset +
2551             (store_lane_gmem_0_Br + 8) * kHeadDim +
2552             store_lane_gmem_0_d;
2553         // LDST128BITS = reinterpret_cast<float4*>
2554         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2555             *reinterpret_cast<float4*>(&R_Z[0][0]);
2556         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2557             *reinterpret_cast<float4*>(&R_Z[1][0]);
2558     }
2559 }
2560 }
2561 }
2562
2563 // =====
2564 // End of notes-v2.cu
2565 // =====
2566 // 本文件覆盖了面试中最高频的 CUDA kernel 考点 (37 个 kernel):
2567 // 基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
2568 // warp+smem+warp0) 优化手段: coalescing, tiling, thread tile, vectorize
,
2569 // pipeline, tensor core Softmax 递进: naive → safe(2-pass) → online(1-
pass,
2570 // FA 基础) GEMM 五层金字塔: tiling → thread tile → vectorize → MMA(TN布
局)
2571 // → WGMMA(warp spec) FlashAttention: split-Q + online softmax + R_S
2572 // 寄存器复用 P0V Bank Conflict: 原理 + PAD 解决方案 + 四步演进 Memory
2573 // Hierarchy: HBM → L2 → L1/SMEM → Register BLAS 布局约定:
2574 // N=col-major(Normal), T=row-major(Transposed), TN=A行B列
2575 // =====
2576 // 本文件覆盖了面试中最高频的 CUDA kernel 考点:
2577 // 基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
2578 // warp+smem+warp0) 优化手段: coalescing, tiling, thread tile, vectorize
,
2579 // pipeline, tensor core Softmax 递进: naive → safe(2-pass) → online(1-
pass,
2580 // FA 基础) GEMM 五层金字塔: tiling → thread tile → vectorize → MMA →
WGMMA
2581 // FlashAttention: tiling + online softmax + recomputation 三板斧
2582 // Bank Conflict: 原理 + PAD 解决方案
2583 // Memory Hierarchy: HBM → L2 → L1/SMEM → Register
2584 // =====

```